工具使用型大语言模型智能体

学习依赖结构与执行状态，而非某一种固定的工具调用顺序

研究报告，依据工作稿 QET v2（Notion，2026-10-13）及其较早版本（2026-10-08）

使用方向键翻页，按 L 键在英文与中文之间切换。

概览

报告结构与状态标签

部分	内容
1 研究动机	为什么单一示范顺序不适合作为训练目标
2 问题设定	任务输入、环境快照与示范轨迹
3 执行图	依赖、非干扰条件与接受执行集合
4 执行状态	把等价历史映射为同一模型输入
5 共享状态计算	在已完成集合上做动态规划
6 训练目标	边际似然、梯度与中间起点
7 算法流程	训练与推理流程
8 定位与现状	相关工作、假设、证据与待解决问题

标签	含义
来自 v2 草稿	所评审草稿中的内容
来自较早草稿	v2 使用、但只在较早草稿中给出的定义
已有结论	已发表或教科书结论，已核对原始出处
推导计算	基于草稿定义的计算，由脚本求得
示意数值	用于讲解的示例数值，不是测量结果
假设	草稿提出、有待检验的主张

超出两版草稿的解读标注为（个人归纳），每页至多出现一次。

方法名称：草稿将该方法称为 QET，本报告只使用该缩写。

来自 v2 草稿

示例：两次读取与一次比较

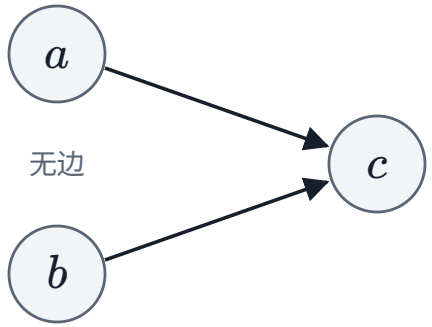
“分别读取数据集 A 和数据集 B，然后比较二者的统计结果。”

$$a = \mathrm{load}(\mathrm{A}), \qquad b = \mathrm{load}(\mathrm{B}), \qquad c = \mathrm{compare}\big(\mathrm{out}(a), \mathrm{out}(b)\big)$$

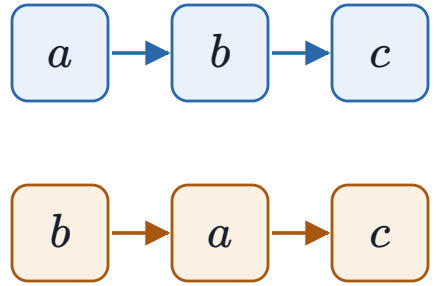
$$a \prec c, \qquad b \prec c$$

在数据快照固定、两次读取互不干扰的条件下，唯一必须满足的顺序是 a 先于 c、b 先于 c。

任务并不要求 a 先于 b，因此 (a, b, c) 与 (b, a, c) 是同一个依赖结构计算的两种执行。



依赖图



两种合法执行

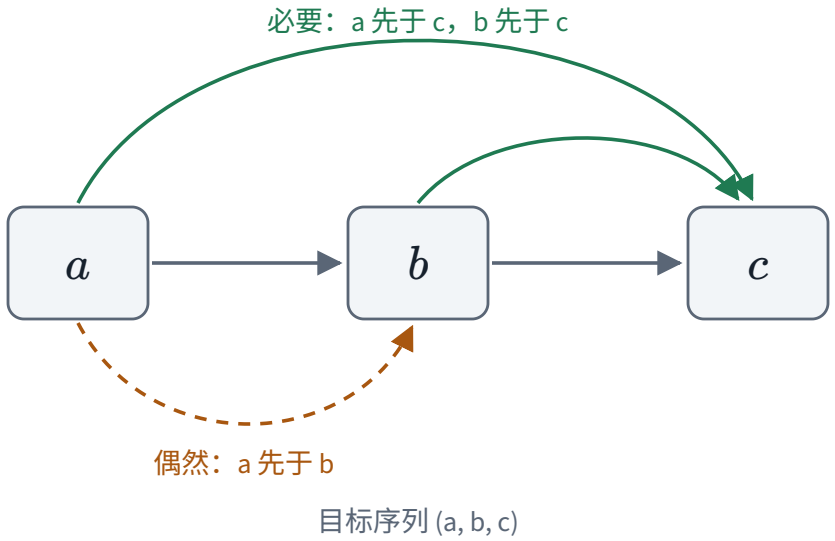
来自 v2 草稿

任务语义与序列化选择

$$\mathcal{L}_{\text{SFT}} = -\log P_\theta(a, b, c \mid x)$$

- **任务语义：**规定哪些动作和数据依赖是必要的，例如两次读取都必须在比较之前完成。
- **序列化选择：**记录示范者恰好先写下了哪个独立动作，这里是先读取 A、再读取 B。

SFT 目标把两者编码成同一种关系，即一段内容出现在另一段之前，因此损失函数无法将二者区分开。



来自 v2 草稿。 对工具智能体而言，问题在于监督信号可能施加了任务本身并不需要的约束。

来自 v2 草稿

对工具智能体的影响

数据与算力

同一计算可能有许多合法顺序；若把每种顺序都当作单独样本来教，数据和算力就会被重复花在同一个计算上。

接续部分完成的工作

智能体可能需要接续其他执行者已部分完成的工作，而从头记住一种顺序并不能保证具备这种能力。

参数可用性

智能体需要知道哪些参数已经可用、哪些前提尚未满足，以及每个值由哪次调用产生。

来自 v2 草稿。 草稿据此认为，执行进度与参数来源应当成为后训练中的显式对象。

学会正确的调用顺序，并不等于学会在多个相互依赖的工具之间管理执行状态。

已有结论

FuncBenchGen 中的证据

FuncBenchGen 在隐藏的函数依赖 DAG 上构造任务，模型需要不断调用函数，直到能够给出目标变量的值。

错误类型	论文中的定义	在 7 个模型中的错误占比
Value Not Yet Known（值尚未获得）	使用了尚未由先前调用或初始输入确定的变量值	66.8% 至 81.3%
Incorrect Value（值错误）	使用了已确定的变量，但取值错误	17.9% 至 33.2%
Function Not Found；Wrong Number of Inputs	接口层面的错误	分别不超过 2.4% 和 0.9%

多数模型能写出语法正确的调用，却难以跟踪哪些值已经存在，以及哪个值应当填入哪个参数。

论文报告，在每次调用后重述所有已知变量值，可使 GPT-5 的成功率从 62.5% 提升到 81.3%。

来自 v2 草稿　示意数值

示例中的两类错误

Value Not Yet Known（值尚未获得）

已完成　A → flight_price = 1200

缺失　B → hotel_price

```
calculate_total(flight_price=1200, hotel_price=?) ×
```

智能体调用了 A 却还没有调用 B，就已尝试执行 C，说明它没有检查 C 的前置条件。

Incorrect Value（值错误）

已完成　A → 1200, B → 600

传入　hotel_price = 900

```
calculate_total(flight_price=1200, hotel_price=900) ×
```

正确的值 600 已经可用，智能体却传入了 900，这个值可能来自环境中另一个工具的返回结果。

两种情况下，变量、产生它的调用与其实际取值之间的对应关系都已经丢失。

两类错误都与执行状态有关：现在可以执行哪个调用，以及它应当接收哪些真实参数。

假设

核心研究假设

假设。 智能体可以学会全部语义合法的工具调用顺序，并能从任意合法的中间状态完成剩余工作。

较早的草稿针对顺序自由度足够大、且可以被可靠识别的任务，给出了可检验的表述。

假设。 在这些条件下，基于合法执行集合的训练，比固定顺序监督、顺序增强或前沿集合监督具有更好的能力与计算成本权衡。

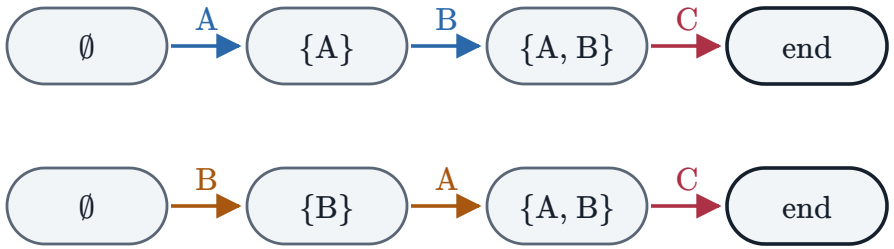
两版草稿都没有报告训练结果；仅有的数值证据是一项自检，用来确认概率递推与梯度的实现彼此一致。

来自 v2 草稿

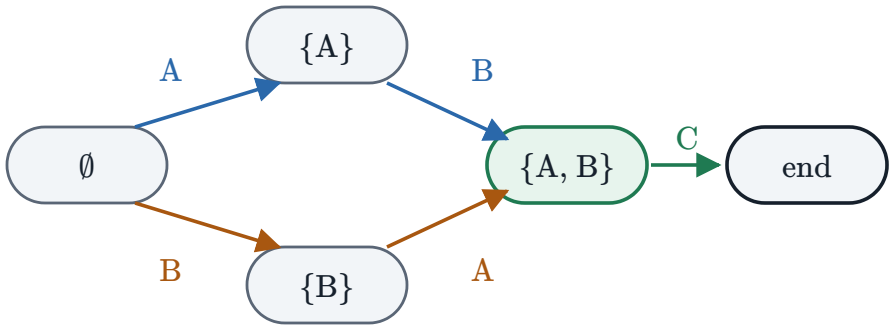
顺序增强的局限

直接的做法是从依赖图中采样更多合法重排，并把每一种重排都作为单独的序列来训练。

- **采样分布：**增强必须指定一个顺序分布，于是模型还要被训练去拟合这个分布。
- **重复计分：**经过一组独立动作后到达同一状态的轨迹，仍被当作不同序列从头到尾分别计分。
- **无法保证接续：**一个能稳定完成 $A \to B \to C$ 的模型，在 B 已由另一个智能体完成时仍可能失败。



逐路径计分：C 被计分两次



共享状态：C 只计分一次

来自 v2 草稿

训练设定的三处改变

	标准 SFT	所提方法（QET）
监督目标	一条示范轨迹 τ	接受执行集合 $\mathcal{C}(G)$
策略输入	完整的时间顺序历史 h_t	规范化执行状态 $H_t = \mathrm{Reduce}(h_t)$
概率计算	每条路径单独计分	在共享的已完成集合上做动态规划
训练起点	仅任务起点	任务起点与合法中间状态

模型学习的是合法解构成的空间，而不是模仿某一条参考轨迹，同时还学习从部分进度继续执行。

v2 草稿，解决方案第 1 至 3 点；第四行来自方法部分关于合法前缀的内容。

来自 v2 草稿　来自较早草稿

任务输入、环境与示范轨迹

$$x = (u, \mathcal{T}, o_0) \qquad (s', o) = \mathrm{Exec}_\zeta(s, a) \qquad \tau = (a_1, o_1, \ldots, a_n, o_n, a^\star_{\mathrm{end}})$$

符号	含义	示例
u	自然语言请求	查询 NYC 到 SG 的机票与酒店价格，然后计算总费用
$\mathcal{T}$	工具说明	`get_flight_price`, `get_hotel_price`, `calculate_total`
o_0	任务开始时智能体可直接获得的信息	草稿中未给出
ζ	固定环境快照：环境保存了什么以及如何恢复	可在每次运行时恢复的价格数据（示意）
s，s'	动作 a 前后的环境真实状态，由执行器保存	–
o	本次调用返回给智能体的观察	`flight_price = 1200`
$a^\star_{\mathrm{end}}$	任务成功完成后的终止动作	返回总费用

快照 ζ 由训练执行器持有，不会作为额外的特权信息提供给策略。

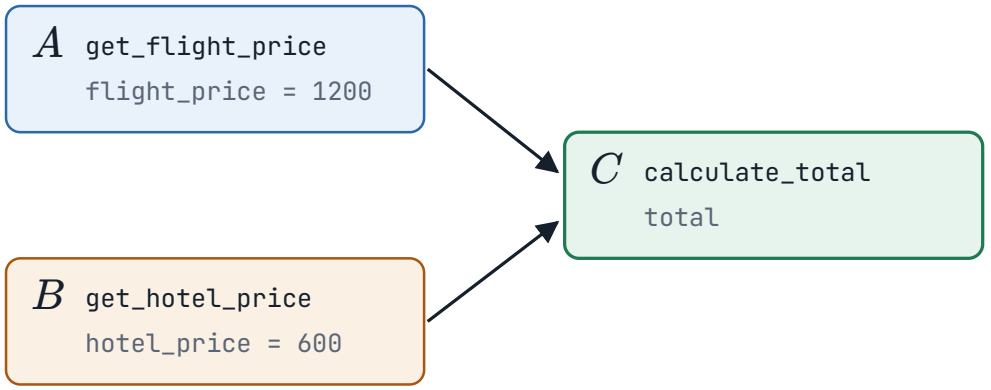
v2 草稿第 2 节；关于特权信息的规定来自较早草稿第 3.1 节。

来自 v2 草稿

目标执行图

$$G = (V, E, \mathcal{A}, \mathcal{O}, \mathcal{K})$$

组成	含义	机票与酒店示例
V	任务需要完成的操作实例	$\{A, B, C\}$
E	必须满足的依赖关系	$\{(A, C), (B, C)\}$
$\mathcal{A}$	每个节点的工具调用与参数绑定	$a_C = \texttt{calculate_total}(p_f, p_h)$
$\mathcal{O}$	每次调用返回的字段	$o_A : \{\texttt{flight_price}\}$
$\mathcal{K}$	执行合法性、数据流与结果验证的约束	$\mathrm{CanExecute}(C) \Leftrightarrow \mathrm{Valid}(o_A) \wedge \mathrm{Valid}(o_B)$



E = {(A, C), (B, C)}；A 与 B 之间无边

节点是操作实例而不是工具类型，因此同一工具被调用两次就对应两个不同的节点。

来自 v2 草稿　已有结论

操作记录与非干扰条件

$$k_v = (a_v, R_v, W_v, P_v, O_v)$$

a_v	实际执行的动作，例如 `get_flight_price()`
R_v	读取集合：读取的数据库字段、文件等状态
W_v	写入集合：修改的状态
P_v	参数来源与前置条件
O_v	工具返回的输出

$$W_u \cap (R_v \cup W_v) = \varnothing, \qquad W_v \cap (R_u \cup W_u) = \varnothing$$

两个条件同时成立时，任一操作写入的内容都不会被另一操作读取或写入，因此交换二者不会改变任何结果。

两个纯查询满足 $W_u = W_v = \varnothing$，必然满足这两个条件，因此机票查询与酒店查询可以交换顺序。

更正。 这正是 Bernstein 条件（1966），它们是安全交换的充分条件而非必要条件，并且要求读写集合完整。

v2 草稿第 3 节把该条件表述为必要条件；较早草稿第 4.4 节正确地称其为充分条件。

来自较早草稿 示意数值

依赖边的来源

操作对	检查	结论
`get_flight_price`, `get_hotel_price`	$W_u = W_v = \varnothing$	可交换，无边
`update_budget(+100)`, `get_budget`	$W_u \cap R_v = \{\mathbf{budget}\}$	写读冲突，保留顺序
`lookup_address`, `send_message`	无写冲突，但发送消息需要地址作为参数	数据依赖，保留顺序

- 仅凭非干扰条件不能判定独立，因为后一个调用可能需要前一个调用的结果作为参数。
- 因此较早的草稿由数据、控制条件、状态作用和任务要求四类边构造 E，并对任何未经验证的动作对保留示范顺序。
- 必须检查所有相关动作对：若 a 写入 x、b 与之无关、c 读取 x，则边 $a \prec c$ 必须保留。

较早草稿第 4.3 至 4.5 节；预算操作对是本报告补充的示意例子。

来自 v2 草稿

线性扩展与接受执行集合

$$\mathrm{LinExt}(G) = \{(A, B, C),\ (B, A, C)\}$$

$$\mathcal{C}(G) = \left\{(a_A, a_B, a_C, a^{\star}_{\mathrm{end}}),\ \ (a_B, a_A, a_C, a^{\star}_{\mathrm{end}})\right\}$$

- G 的线性扩展是全部节点的一种排列，其中每条边的起点都排在终点之前；$\mathrm{LinExt}(G)$ 是所有这类排列的集合。
- 接受执行集合 $\mathcal{C}(G)$ 把每个线性扩展写成完整的工具调用序列，并在末尾加上终止动作。
- 模型生成的序列只要属于 $\mathcal{C}(G)$，就算作正确执行，即使它与示范轨迹不同。
- 不确定的动作对会保留示范顺序，因此 $\mathcal{C}(G)$ 可能小于全部真正等价的执行集合。

v2 草稿第 3 节，草稿称该集合为执行接受类；最后一点来自较早草稿第 4.6 节。

来自 v2 草稿

从单一目标到目标集合

$$\mathcal{L}_{\text{SFT}} = -\log P_\theta(A, B, C \mid x)$$

$$\mathcal{L} = -\log \sum_{\tau \in \mathcal{C}(G)} P_\theta(\tau \mid x)$$

- 标准 SFT 只提高示范轨迹 (A, B, C) 的概率，对 (B, A, C) 不给任何正向信号。
- 集合目标只要模型把概率放在任一接受执行上，就会给出正向训练信号。
- 模型不再被强制沿着示范者恰好事先确定的那条路径执行。

训练目标从模仿某一个标准答案，转变为学习一个合法解空间。

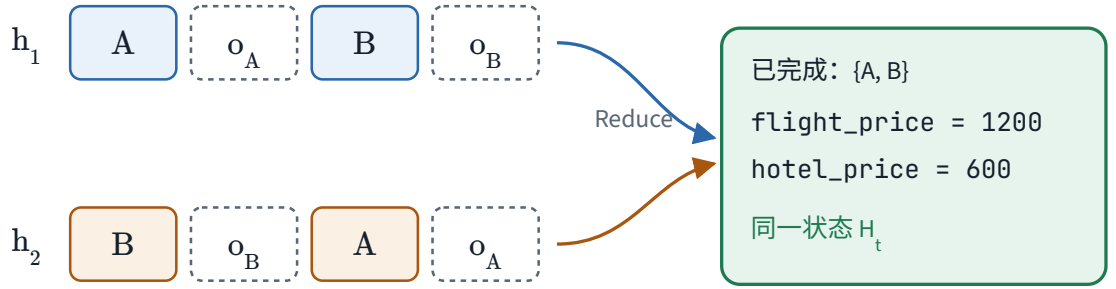
v2 草稿，解决方案第 1 点。草稿中第二条轨迹的下标重复，此处应为第二条。

来自 v2 草稿

Transformer 策略对历史顺序的依赖

$$h_1 = (A, o_A, B, o_B), \qquad h_2 = (B, o_B, A, o_A)$$

- 若 A、B 相互独立且返回结果相同，两段历史描述的任务进度完全一样。
- Transformer 读入的 token 顺序和位置不同，因此仍可能给出 $P_\theta(C \mid h_1) \neq P_\theta(C \mid h_2)$。
- 下一步动作的概率一旦不同，两段历史就不能合并为同一状态来做精确的概率计算。



token 顺序不同，任务进度相同

s_t	第 t 步的环境真实状态，由执行器保存
h_t	截至第 t 步的完整交互历史
H_t	提供给 LLM 的规范化观测状态

v2 草稿第 1 节与第 3.1 节。v2 在引言中也用 s_t 表示规范化状态；本报告统一使用 H_t。

来自 v2 草稿 来自较早草稿

Reduce 与 Render

$$\mathrm{Reduce}(h_1) = \mathrm{Reduce}(h_2) = H_t$$

$$\pi_\theta(a \mid x, H_t) = p_\theta\big(a \mid x, \mathrm{Render}(H_t)\big)$$

- Reduce 把任务进度相同的所有历史映射为同一个规范化状态，该状态只由已完成动作及其观察构成。
- Render 把该状态写成固定文本，因此两段历史得到完全相同的提示词和下一步动作分布。
- Reduce 只能使用已发生的动作、实际观察和运行时可访问的契约，不能使用目标图或未来动作。

```
Completed operations:
- flight_query: price = 1200
- hotel_query: price = 600

Available values:
- flight_price: 1200, source = flight_query
- hotel_price: 600, source = hotel_query
```

两段历史之后的渲染状态，h_1 与 h_2 得到的文本完全相同。

来自较早草稿。 规范化只去除经认证与任务无关的顺序；对同一变量的覆盖写入以及任务要求的先后顺序都会保留。

v2 草稿第 3.1 节；对 Reduce 的约束来自较早草稿第 5.2 与 5.3 节。

来自 v2 草稿 来自较早草稿

完整动作的概率

$$\log \pi_\theta(a \mid x, H) = \sum_{\ell=1}^{L} \log p_\theta(a_\ell \mid x, H, a_{<\ell})$$

```
{
  "tool": "calculate_total",
  "arguments": {
    "flight_price": 1200,
    "hotel_price": 600
  }
}
```

节点 C 的完整动作 a，包括结尾的结束标记。

- 动作概率覆盖调用中的每个 token，包括工具名称、参数、格式以及明确的结束标记。
- 传入 `hotel_price = 900` 的调用是另一个 token 序列，因此训练目标不会奖励这个错误参数。
- 工具返回内容作为观察输入而不是预测目标，因此在确定性的接受路径上不需要额外的环境概率因子。

来自较早草稿。 得分使用策略的实际概率，因为若只在正确动作之间重新归一化，就会丢弃模型分配给错误动作的概率。

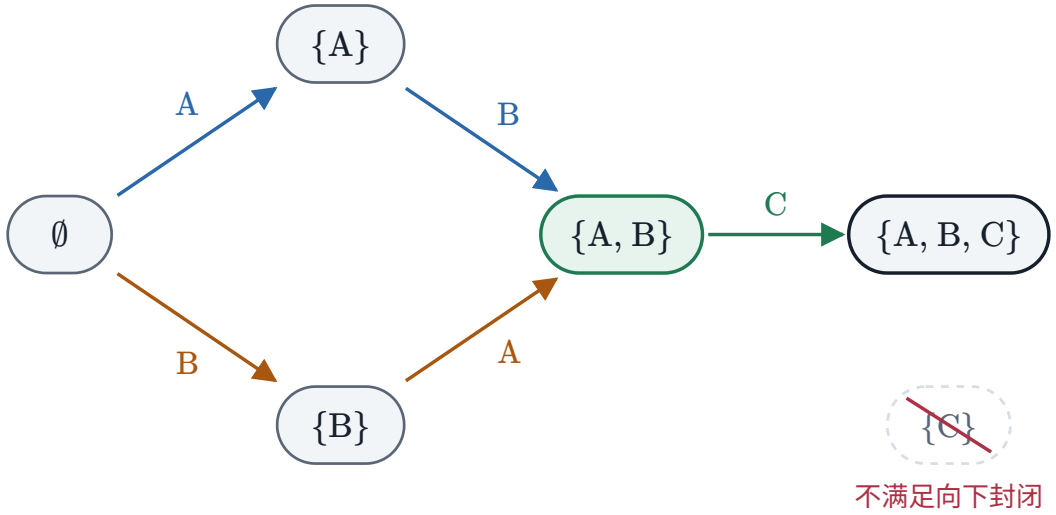
来自 v2 草稿　已有结论

已完成集合与序理想

$$v \in I \implies \mathrm{pa}_G(v) \subseteq I$$

$$\mathcal{J}(G) = \{\varnothing, \{A\}, \{B\}, \{A, B\}, \{A, B, C\}\}$$

- 已完成集合 $I \subseteq V$ 列出已经完成的操作；只有包含其中每个操作的全部前驱时，它才合法。
- 这类向下封闭的集合称为序理想，它们按包含关系构成一个格，作为训练时的状态空间。
- $\{C\}$ 不合法，因为在父节点 A、B 都未完成时，C 不可能已经完成。



合法已完成集合 $\mathcal{J}(G)$，按包含关系排列

来自 v2 草稿 来自较早草稿

状态共享的条件

$$H_I = \mathrm{State}(x, I)$$

只有当所有到达 I 的接受路径都产生相同的规范化状态，从而产生相同的模型输入时，合并才是精确的。

- **接受路径确定：**接受路径上的工具结果是确定的，或者具有经过验证的等价规范形式。
- **归约与顺序无关：**交换两个经认证独立的动作不会改变归约后的状态，记为 $\mathrm{Reduce}(h\, a\, o_a\, b\, o_b) = \mathrm{Reduce}(h\, b\, o_b\, a\, o_a)$。
- **无隐藏记忆：**策略在回合之间不保留额外的草稿文本，因此相同的 H_I 总是产生相同的模型输入。

说明。 共享只是节省计算：到达 I 的不同路径各自保留自己的概率，这些概率被相加，而不是被假设为相等。

来自 v2 草稿

就绪集合与单步得分

$$\mathcal{R}_G(I) = \{v \in V \setminus I : \ \mathrm{pa}_G(v) \subseteq I\}$$

$$w_\theta(I, v) = \pi_\theta(a_v \mid x, H_I), \qquad e_\theta(V) = \pi_\theta(a^\star_{\mathrm{end}} \mid x, H_V)$$

I	$\mathcal{R}_G(I)$	所需得分
$\varnothing$	$\{A, B\}$	$w_\theta(\varnothing, A),\ w_\theta(\varnothing, B)$
$\{A\}$	$\{B\}$	$w_\theta(\{A\}, B)$
$\{B\}$	$\{A\}$	$w_\theta(\{B\}, A)$
$\{A, B\}$	$\{C\}$	$w_\theta(\{A, B\}, C)$
$\{A, B, C\}$	$\varnothing$	$e_\theta(V)$

- 就绪集合包含所有前提均已完成、但自身尚未执行的操作，因此其中每个操作现在都可以执行。
- $w_\theta(I, v)$ 是模型在状态 I 下生成的调用恰好为 a_v 的概率。
- $e_\theta(V)$ 是所有操作完成之后正确结束任务的概率。
- 本例共需 6 个得分：格中每条边一个，再加上终止动作一个。

来自 v2 草稿　来自较早草稿

完成概率的递推

$$B_\theta(I) = \sum_{v \in \mathcal{R}_G(I)} w_\theta(I, v)\, B_\theta(I \cup \{v\}), \qquad B_\theta(V) = e_\theta(V)$$

项	含义
$B_\theta(I)$	从 I 出发，按任一接受顺序完成全部剩余操作并正确结束的概率
$w_\theta(I, v)$	在状态 I 下接着执行 v 的概率
$B_\theta(I \cup \{v\})$	执行 v 之后的完成概率
$B_\theta(V) = e_\theta(V)$	边界条件：所有操作均已完成，只剩终止动作

每条接受后缀都恰好以一个就绪动作开头，因此对就绪动作求和时，每条后缀只被计算一次。

更正。 v2 定义了 $e_\theta(V)$ 但没有写出边界条件，较早的草稿则明确给出了 $B_\theta(V) = e_\theta(V)$。

v2 草稿第 3 节；正确性论证及其对动作标签无歧义的要求来自较早草稿第 7.3 节。

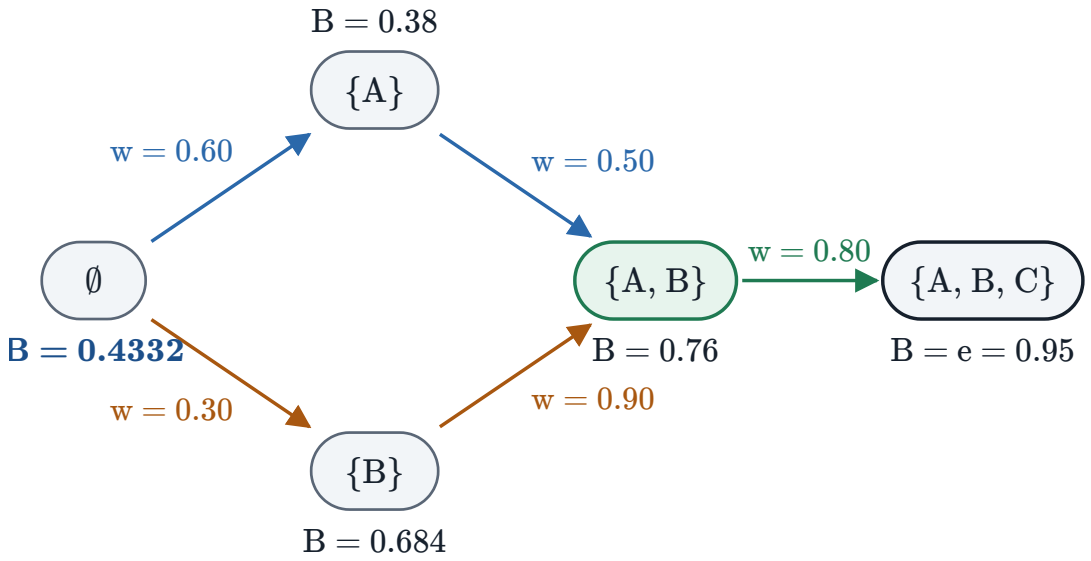
来自较早草稿　示意数值

逐步计算示例

1. $B(V) = e(V) = 0.95$
2. $B(\{A, B\}) = 0.80 \times 0.95 = 0.76$
3. $B(\{A\}) = 0.50 \times 0.76 = 0.38$，$B(\{B\}) = 0.90 \times 0.76 = 0.684$
4. $B(\varnothing) = 0.60 \times 0.38 + 0.30 \times 0.684 = 0.228 + 0.2052 = 0.4332$

总概率等于两条接受路径的概率之和，而经过 $\{A, B\}$ 的共享后缀只计分一次。

在 $\varnothing$ 处，剩余的 0.10 概率落在接受集合之外的动作上，例如过早调用 C。



来自 v2 草稿　推导计算

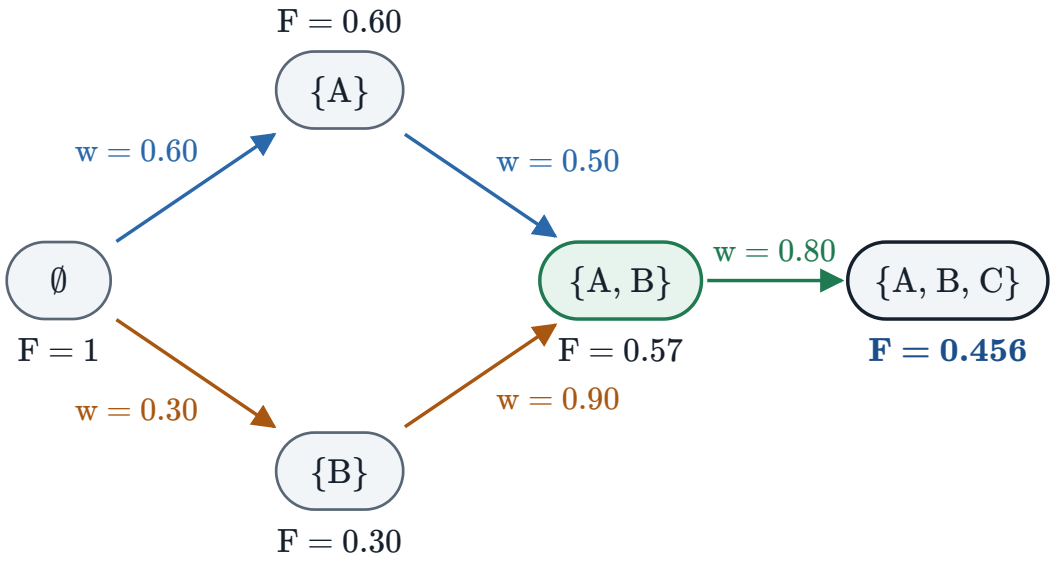
同一求和的前向视角

$$F(I \cup \{v\}) \mathrel{+}= F(I)\, w_\theta(I, v), \qquad F(\varnothing) = 1$$

1. $F(\{A\}) = 0.60, \quad F(\{B\}) = 0.30$
2. $F(\{A, B\}) = 0.60 \times 0.50 + 0.30 \times 0.90 = 0.30 + 0.27 = 0.57$
3. $F(V) = 0.57 \times 0.80 = 0.456$
4. $F(V)\, e_\theta(V) = 0.456 \times 0.95 = 0.4332 = B_\theta(\varnothing)$

$F(I)$ 是沿接受路径到达 I 的总概率；v2 正是用这种前向形式引入方法的。

两个方向都得到 0.4332，这正是有向无环图上路径求和的前向–后向恒等式。



更正。 v2 引言中的 F 省略了终止动作，乘以 $e_\theta(V)$ 之后即与 $B_\theta(\varnothing)$ 一致。

推导计算 已有结论

共享状态的作用

依赖结构	接受顺序数	有序前缀数	已完成集合数	格的边数
3 个独立操作	6	16	8	12
10 个独立操作	3,628,800	9,864,101	1,024	5,120
20 个独立操作	2.43×10^{18}	6.61×10^{18}	1,048,576	10,485,760
4 条长度为 5 的链	11,732,745,024	35,899,051,101	1,296	4,320
1 条长度为 20 的链	1	21	21	20

- n 个独立操作有 $n!$ 种顺序，但只有 2^n 个已完成集合；10 个操作对应 3,628,800 种顺序和 1,024 个状态。
- 有序前缀数即前缀树的节点数，因此仅靠前缀缓存无法合并“先 A 后 B”与“先 B 后 A”（个人归纳）。
- 状态数仍可能指数增长；正如 v2 所指出的，只有在确实存在可合并的状态时，才会有压缩收益。

计数由构建脚本得出；1,296 与较早草稿第 10.1 节一致。线性扩展计数是 #P 完全问题（Brightwell 与 Winkler，1991）。

来自 v2 草稿 来自较早草稿

接受执行集合上的边际似然

$$\mathcal{L}_{\text{root}}(\theta) = \mathbb{E}_{(x,G)\sim\mathcal{D}}\big[-\log B_\theta(\varnothing)\big]$$

$$B_\theta(\varnothing) = P_\theta\big(\mathcal{C}(G)\big) = \sum_{\sigma\in\mathrm{LinExt}(G)} p_\theta(\sigma)$$

- 根状态损失是智能体通过任一接受执行完成任务的概率的负对数。
- 在逐步计算示例中，这一个任务的根状态损失为 $-\ln 0.4332 = 0.837$ nat。
- 若每条接受执行都能通过任务检查，则 $B_\theta(I)$ 是从 I 出发任务成功概率的下界。
- 因此，该目标优化的是一个经过认证的成功事件，而不是所有可能完成任务的策略。

v2 草稿，类边际目标；下界的说明来自较早草稿第 7.4 节。

来自 v2 草稿 推导计算

两个示例模型上的三种目标

$$\mathcal{L}_{\mathrm{SFT}} = -\log p_\theta(ABC)$$

$$\mathcal{L}_{\mathrm{aug}} = -\tfrac{1}{2}\left[\log p_\theta(ABC) + \log p_\theta(BAC)\right]$$

$$\mathcal{L}_{\mathrm{root}} = -\log\left[p_\theta(ABC) + p_\theta(BAC)\right]$$

两个模型到达完成状态的总概率都是 0.50，因此边际目标认为二者等价。

	模型 1	模型 2
$p_\theta(ABC)$	0.45	0.25
$p_\theta(BAC)$	0.05	0.25
合法路径总概率	0.50	0.50
$\mathcal{L}_{\mathrm{root}}$	0.693	0.693
$\mathcal{L}_{\mathrm{aug}}$	1.897	1.386
$\mathcal{L}_{\mathrm{SFT}}$	0.799	1.386

- 顺序增强更偏好模型 2，因为它还会奖励两种合法顺序之间的均匀分配。
- 在 (A, B, C) 上做 SFT 则偏好模型 1，而边际目标对同样合法的顺序没有偏好。

概率及前两行损失来自 v2；SFT 一行为对比而推导。损失单位为 nat。

来自较早草稿 已有结论

顺序增强损失的分解

$$\mathbb{E}_{\sigma\sim\omega}\big[-\log p_\theta(\sigma)\big] = -\log P_\theta\big(\mathcal{C}(G)\big) + \mathcal{H}(\omega) + D_{\mathrm{KL}}\big(\omega \,\big\|\, p_\theta(\cdot \mid \mathcal{C}(G))\big)$$

- 在按 ω 抽取的顺序上训练，等于边际损失，加上 ω 的熵，再加上一个拟合 ω 的项。
- 当 ω 在 M 个顺序上均匀分布时，$\mathcal{H}(\omega) = \log M$ 为常数，因此只有 KL 项能区分两个模型。
- 先对概率取平均再取对数，只会给边际损失加上常数 $\log M$，因此梯度不变。

$$-\log\Big(\tfrac{1}{M}\sum_\sigma p_\theta(\sigma)\Big) = -\log\sum_\sigma p_\theta(\sigma) + \log M$$

	$\mathcal{L}_{\mathrm{aug}}$	$\mathcal{L}_{\mathrm{root}}$	$\mathcal{H}(\omega)$	D_{KL}
模型 1	1.897	0.693	0.693	0.511
模型 2	1.386	0.693	0.693	0.000

每一行都满足 $\mathcal{L}_{\mathrm{aug}} = \mathcal{L}_{\mathrm{root}} + \mathcal{H}(\omega) + D_{\mathrm{KL}}$，其中 ω 为均匀分布。

来自较早草稿。 额外的 KL 项未必有害，它也可能起到正则化作用，因此孰优孰劣需要由实验回答。

较早草稿第 8.2 节式 (21)；v2 只给出了 log M 恒等式，没有给出分解。

来自较早草稿 推导计算

后验加权梯度

$$\nabla_\theta \log \sum_{\sigma \in \mathcal{C}} p_\theta(\sigma) = \mathbb{E}_{\sigma \sim p_\theta(\cdot \mid \mathcal{C})}\left[\nabla_\theta \log p_\theta(\sigma)\right]$$

$$q_\theta(v \mid I, G) = \frac{w_\theta(I, v)\, B_\theta(I \cup \{v\})}{B_\theta(I)}$$

- 边际似然的梯度就是普通的对数似然梯度，只是每条接受路径按其在成功条件下的后验概率加权。
- 在逐步计算示例中，第一个动作的后验为 $q(A \mid \varnothing) = 0.60 \times 0.38 / 0.4332 = 0.526$ 与 $q(B \mid \varnothing) = 0.474$。
- 它们不同于策略自身的比例 $0.60/0.90 = 0.667$，因为先做 B 之后完成的可能性更高。

解读。 顺序增强按固定的 ω 给每种顺序加权，而边际梯度跟随模型自身在成功条件下的偏好（个人归纳）。

q_θ 的定义见较早草稿第 9.1 节；v2 在算法 1 中使用了它，但没有给出定义。

来自较早草稿 推导计算

转移权重与两阶段更新

$$m_\theta(I) \leftarrow \mu_G(I), \qquad m_\theta(I \cup \{v\}) \mathrel{+}= m_\theta(I)\, q_\theta(v \mid I, G)$$

$$\rho_\theta(I, v) = m_\theta(I)\, q_\theta(v \mid I, G)$$

$$\widetilde{\mathcal{L}}_G = -\sum_{I,v} \operatorname{sg}\left[\rho_\theta(I, v)\right] \log w_\theta(I, v) - \log e_\theta(V)$$

- $\rho_\theta(I, v)$ 是成功后验在状态 I 下选择动作 v 的期望次数。
- 停止梯度先固定权重，再重新计算 token 对数概率，从而在当前参数处得到精确的边际似然梯度。
- 该代理损失的数值不同于边际损失，因此训练日志应当记录由动态规划计算的损失值。

转移	ρ_θ
$(\varnothing, A)$	0.263
$(\varnothing, B)$	0.237
$(\{A\}, B)$	0.513
$(\{B\}, A)$	0.487
$(\{A, B\}, C)$	1.000
合计	2.5

起点权重取 $\mu(\varnothing) = 0.5$、$\mu(\{A\}) = \mu(\{B\}) = 0.25$，与较早草稿一致；权重之和等于 $\mathbb{E}[n - |S|]$。

较早草稿第 9.2 至 9.5 节及第 13.1 节；v2 的算法 1 计算了这些量，但没有给出定义。

来自 v2 草稿　示意数值

仅从根状态训练的局限

一个只掌握一种顺序、很少能从其他合法状态完成任务的策略，仍然可以得到很低的根状态损失。

	示例策略	顺序坍缩策略
$w(\varnothing, A),\ w(\varnothing, B)$	0.60, 0.30	0.85, 0.05
$w(\{A\}, B),\ w(\{B\}, A)$	0.50, 0.90	0.90, 0.10
根状态损失 $-\log B(\varnothing)$	0.837	0.536
在 B 之后接续：$-\log B(\{B\})$	0.380	2.577

- 顺序坍缩策略的根状态损失更低，但从 $\{B\}$ 出发时，它完成任务的概率只有 0.076。
- 仅从根状态训练的目标看不到这一缺陷，因为没有任何训练样本从另一个智能体已完成 B 的状态开始。

论证来自 v2 与较早草稿第 8.3 节。顺序坍缩策略使用示意数值（个人归纳）；两者均取 $w(\{A, B\}, C) = 0.80$、$e(V) = 0.95$。

来自 v2 草稿

从中间状态开始训练

$$\mu_G = (1-\lambda)\,\delta_{\varnothing} + \lambda\,\mu_{\text{cut}} \qquad \widehat{\mu}_G = (1-\lambda)\,\delta_{\varnothing} + \frac{\lambda}{K}\sum_{k=1}^{K}\delta_{I_k}$$

$$\mathcal{L}_{\text{QET}}(\theta) = \mathbb{E}_{(x,G)\sim\mathcal{D}}\,\mathbb{E}_{I\sim\mu_G}\big[-\log B_\theta(I)\big]$$

- 每次更新先采样一个任务，再从 μ_G 中采样一个合法起始状态 I，并训练从该状态完成任务。
- λ 决定非根起点的权重，并用 K 个采样得到的截断状态代替对全部合法已完成集合的枚举。
- H_I 只由 I 中动作的实际观察构造，不使用示范中后续步骤的信息。

起点 I	已完成	剩余工作
$\varnothing$	无	$A \to B \to C$ 或 $B \to A \to C$
$\{A\}$	A	$B \to C$
$\{B\}$	B	$A \to C$
$\{A, B\}$	A, B	C

v2 草稿，合法前缀覆盖；关于观察的规定来自 v2 的算法 1。

来自较早草稿　示意数值　推导计算

中间起点的作用

$$\mu(\varnothing) = 0.5, \qquad \mu(\{A\}) = \mu(\{B\}) = 0.25$$

	示例策略	顺序坍缩策略
根状态损失 $-\log B(\varnothing)$	0.837	0.536
混合损失 $\mathcal{L}_{\mathrm{QET}}$	0.755	1.007

$$0.5 \times 0.837 + 0.25 \times 0.968 + 0.25 \times 0.380 = 0.755$$
$$0.5 \times 0.536 + 0.25 \times 0.380 + 0.25 \times 2.577 = 1.007$$

- 在两个单动作起点上各放四分之一的权重之后，示例策略的损失反而更低。
- 若实际接续起点分布满足 $\nu_G(I) \le \kappa\, \mu_G(I)$，则期望失败概率满足 $\mathbb{E}_{\nu_G}[1 - B_\theta(I)] \le \kappa\, \mathbb{E}_{\mu_G}[-\log B_\theta(I)]$。
- 该界由 $1 - z \le -\log z$ 推出，只对所覆盖的任务与合法接续状态成立。

起点权重取自较早草稿第 13.1 节；覆盖界来自其第 12.3 节命题 3。损失单位为 nat。

来自 v2 草稿

训练流程（算法 1）

离线准备

1. 记录动作、参数来源、观察、状态版本与控制条件
2. 检查所有相关动作对，构造保守的目标图 G
3. 验证动作事件无歧义、示范成功与交换证据
4. 按底层任务与模板划分训练、验证与测试集

每次参数更新

1. 采样任务及其固定目标图，再从 μ_G 采样根状态与合法前缀
2. 仅用已完成动作的实际观察构造 H_I
3. 小图：完整的格。大图：有界展开
4. 计算 B 或明确标记的下界 B^-，再计算 q、m、ρ
5. 固定后验权重，重算加权的 token 对数概率
6. 跨微批次累积梯度，然后执行一次优化器更新

- 只训练 LLM 或其参数高效适配器，图编译器与状态接口保持固定。
- 所有对比方法共享格式预热、工具说明、状态模板和可比的优化配置，以便单独识别目标函数的作用。

来自较早草稿

精确计算与有界近似

$$B_\theta^-(I) \;\le\; B_\theta(I) \;\le\; B_\theta^+(I)$$

$$0 \;\le\; -\log B_\theta^-(I) + \log B_\theta(I) \;\le\; \log \frac{B_\theta^+(I)}{B_\theta^-(I)}$$

$$O(w\,N_G), \qquad N_G = |\mathcal{J}(G)| \le \prod_{j=1}^{k} (\ell_j + 1)$$

说明。 v2 在算法 1 中提到了 B^-、B^+、C 与 ε，但只有较早草稿的第 10 节给出了定义。

- 小图使用完整的格，大图只在预算 C 内展开其中一部分。
- 未展开状态取 $B^- = 0$、$B^+ = 1$，再用同一递推即可得到 $B_\theta(I)$ 的确定上下界。
- 每个采样起点先播种一条完整的接受后缀，使 B^- 保持为正，损失保持有限。
- 预算耗尽时，训练使用真实损失的上界 $-\log B^-$，并须如实标注。
- 精确计算需要 $O(w\,N_G)$ 次动作计分，其中 w 是最大就绪集合的大小，ℓ_j 是 G 的某个链分解中各条链的长度。

较早草稿第 10.1 至 10.5 节。

来自 v2 草稿　来自较早草稿

推理流程（算法 2）

循环，直到终止动作或任务预算耗尽

1. 初始化 H_0
2. 根据 x 与 $\mathrm{Render}(H_t)$ 生成完整动作 a_t
3. 若 a_t 为终止动作，则输出结果
4. 否则执行该调用，获得观察 o_t
5. 用运行时信息更新 H_{t+1}

- 推理时智能体只看到 x 与 $\mathrm{Render}(H_t)$，不会访问 G、完成格、B_θ 或 q_θ。
- 训练与推理使用同一套 Reduce 与 Render 实现，因此模型看到的状态格式始终一致。

注意。 类概率是在采样策略下定义的，贪心解码的表现可能截然不同。设十个正确首动作的概率各为 0.09，一个错误首动作的概率为 0.10。此时采样以 0.90 的概率选中正确首动作，而贪心解码总是选中错误动作。

v2 草稿，算法 2；访问限制与解码示例来自较早草稿第 11.2 与 11.3 节。

已有结论 来自较早草稿

相关方法

工作	主要内容	较早草稿所述的区别
顺序中心增强（He 等，2025）	依据依赖 DAG 生成合法重排，作为额外训练数据	优化接受集合的边际概率，并覆盖中间状态
智能体轨迹去线性化（Li 等，2026）	从线性轨迹推断潜在偏序，用于更高效的执行	把已认证的偏序作为监督信号，用于更新 LLM 策略
图生成中的顺序边际化（Chen 等，ICML 2021）	把节点顺序视为潜变量，训练其变分界	处理交互环境中的动作、观察与可执行性
动态 oracle（Goldberg 与 Nivre，TACL 2013）	在结构化解析器训练中允许多个正确转移	对整条接受后缀计分，并精确合并状态
LLMCompiler（Kim 等，ICML 2024） †	在推理时规划并并行执行相互独立的函数调用	改变的是训练目标，而不是执行器

较早的草稿只把创新点限定为三者的组合：经认证的执行等价、在线状态接口，以及带中间起点的完成似然训练。

它明确不主张自己是首个使用偏序、首个对顺序求和或首个允许多个正确下一步的工作。

前四行依据较早草稿第 2.2 节；† 表示本报告补充的一行（个人归纳）。

已有结论

并发理论与序列建模中的基础

结论	内容	在本工作中的作用
Bernstein 条件（1966）	读写集合互不相交时，两个程序段可以按任意顺序执行	认证哪些工具调用可以交换
偏序归约（Godefroid，1996）	对每一类等价交错只探索一个代表	同样的等价关系，在此用于定义训练目标而非剪枝搜索
序理想与线性扩展（Brightwell 与 Winkler，1991）	线性扩展计数是 #P 完全问题，而在序理想上的动态规划可以精确求解	说明为何以已完成集合构成的格作为状态空间
CTC（Graves 等，ICML 2006）	用前向–后向递推对同一标签序列的所有对齐求和	同样的动态规划模式，只是把对齐换成了已完成集合

前两列中的结论均为已有结论，最后一行与 CTC 的类比属于解读（个人归纳）。

出处见参考文献页；草稿引用了偏序方法（Godefroid），但没有引用 Bernstein 或 CTC。

来自较早草稿 来自 v2 草稿

假设与适用范围

- **接受路径确定：**在快照下，接受动作返回确定的结果；无效输入仍可能产生随机错误，FuncBenchGen 即是如此。
- **有限示范：**循环按实际发生的次数展开，目标图不覆盖其他循环次数。
- **保守构图：**错误的独立性判断会把非法执行加入正向监督，较早的草稿把它列为最高优先级的风险。
- **只含成功前缀：**该目标只覆盖合法的部分执行，不学习工具失败或错误写入之后的恢复。
- **每个任务一个接受集合：**使用其他工具或算法的解不属于 $\mathcal{C}(G)$，因此评估必须以环境中的任务成功为准。

已有结论 假设

各项主张的现状

已有结论

- FuncBenchGen 的错误统计，以及重述已知变量值带来的收益。
- Bernstein 条件作为安全交换的充分判据。
- 在序理想上用动态规划求路径概率和的精确性。
- 边际似然梯度恒等式与顺序增强损失的分解。

草稿中的推导

- 在确定性、状态一致与动作无歧义条件下递推的正确性。
- 上下界 $B^- \le B \le B^+$ 及其对数间隙界。
- 接续分布受 μ_G 控制时的覆盖界。
- 在 30 个随机 DAG 上的自检：差异分别不超过 5.6×10^{-17} 与 2.2×10^{-16}。

假设，尚未检验

- H1：去除无关的顺序约束有助于有限预算下的学习。
- H2：从中间状态训练能提高接续能力。
- H3：共享状态使边际训练的成本可以接受。
- H4：认证质量决定方法是否有效。
- H5：收益随任务真实的顺序自由度增大。

目前还没有任何 LLM 训练或基准测试结果，因此本报告中所有关于性能的表述都仍是假设。

较早草稿第 12、13.2 与 14.1 节；FuncBenchGen 表 3 与第 4.6.1 节。

来自较早草稿

计划中的基线与控制变量

基线	定义	控制的因素
Canonical-State SFT	与所提方法相同的状态接口，只训练一种顺序	状态接口带来的收益
Order-Augmented SFT	使用保持依赖的重排作为额外序列	多看顺序带来的收益
Prefix-Matched SFT	使用与所提方法相同的起始前缀	额外状态覆盖带来的收益
前沿集合监督	$-\log \sum_{v \in \mathcal{R}_G(I)} \pi_\theta(a_v \mid x, H_I)$	对整条后缀做边际化是否必要
采样顺序边际化	对采样并去重后的接受顺序求概率和	共享状态算法的价值

- 所有基线都获得相同的变量表，因为 FuncBenchGen 表明，仅重述已知变量值就会改变成功率。
- 各方法的参数接口也必须一致，要么都用句柄，要么都直接输出数值，否则任务本身就变了。

较早草稿第 14.5 与 14.6 节。

来自较早草稿

评价指标与证伪标准

计划的测试环境包括：加入作用契约与可恢复快照的扩展版 FuncBenchGen，以及经过插桩的 AppWorld 任务子集。

评价指标

- 真实环境中的任务成功率，由程序化的状态检查判定。
- 从合法前缀接续的成功率，后端需恢复到对应状态。
- 辅助错误：参数来源、使用未知变量、冗余调用、提前终止与副作用。
- 完整成本：训练 token 数、GPU 时间、计分次数、环境调用与构图开销。

证伪标准

- 简单方法已足够：在相同预算下，规范顺序或顺序增强达到同等表现。
- 局部监督已足够：前沿集合监督以低得多的成本达到同等效果。
- 可用结构太少：大多数真实任务图退化为链，或认证成本过高。
- 表示占主导：收益几乎全部来自变量表或规范化状态。
- 解码落差：似然的改善不能转化为实际部署解码策略下的成功。

较早草稿第 14.2、14.3、14.7 与 14.10 节。

来自较早草稿 来自 v2 草稿

待解决的问题

- **大规模认证：**真实工具可靠的读写契约难以获得，其覆盖范围决定了哪些任务能进入训练。
- μ_{cut} **的选择：**v2 没有规定截断分布，较早的草稿则先采样截断深度，再逐步采样就绪节点直到该深度。
- **后验集中：**若模型初始时偏好某一种顺序，成功后验就会集中到这一顺序上，前缀覆盖只能部分缓解这一问题。
- **规范化状态的成本：**渲染后的状态文本未必共享前缀，因此跨状态复用键值缓存并非没有代价。
- **随机工具与多种策略：**随机的工具输出与多种解法需要显式建模结果分支，或者对多个接受集合取并集。

总结

- 序列目标把必要依赖与偶然顺序混在一起，因此所提方法的训练目标是接受执行集合 $\mathcal{C}(G)$。
- 规范化执行状态让等价历史成为完全相同的模型输入，从而使跨历史的概率共享成为精确计算。
- 在已完成集合上的动态规划给出边际似然，其梯度按成功后验对每条路径加权。
- 从合法中间状态开始训练针对的是接续能力，而仅从根状态训练无法观察到这类失败。

假设。 这些改变能否在固定预算下提升智能体仍未经检验，其结论取决于能否可靠地认证动作之间的独立性。

符号表

符号	含义
$x = (u, \mathcal{T}, o_0)$	任务输入：请求、工具、初始观察
$\zeta,\ \mathrm{Exec}_\zeta$	环境快照及其执行器
$\tau,\ a^\star_{\mathrm{end}}$	示范轨迹与终止动作
$G = (V, E, \mathcal{A}, \mathcal{O}, \mathcal{K})$	目标执行图
$k_v = (a_v, R_v, W_v, P_v, O_v)$	操作记录
$\mathrm{LinExt}(G),\ \mathcal{C}(G)$	线性扩展；接受执行集合
$h_t,\ H_t$	原始历史；规范化状态
$\mathrm{Reduce},\ \mathrm{Render}$	历史到状态的映射；状态到文本的映射

符号	含义
$I,\ \mathcal{J}(G)$	已完成集合；全部序理想
$\mathrm{pa}_G(v),\ \mathcal{R}_G(I)$	v 的父节点；I 处的就绪集合
$w_\theta(I, v),\ e_\theta(V)$	单步得分；终止得分
$B_\theta(I),\ F(I)$	完成概率；前向概率
$q_\theta,\ m_\theta,\ \rho_\theta$	成功后验；占用量；转移权重
$\mu_G,\ \lambda,\ K$	起点分布；非根权重；采样数
$B_\theta^-,\ B_\theta^+,\ C,\ \varepsilon$	下界与上界；预算；误差阈值
$\mathcal{L}_{\mathrm{root}},\ \mathcal{L}_{\mathrm{QET}}$	根状态损失；含中间起点的损失

符号沿用 v2 方法部分与较早草稿第 3.3 节。

参考文献

1. Maekawa, S., Hassell, J., Pezeshkpour, P., Mitchell, T., Hruschka, E. Towards Reliable Benchmarking: A Contamination Free, Controllable Evaluation Framework for Multi-step LLM Function Calling. ICLR 2026. arXiv:2509.26553.
2. He, Q., He, Q., Liang, J., Xiao, Y., Zhou, W., Sun, Z., Yu, F. Order Doesn't Matter, But Reasoning Does: Training LLMs with Order-Centric Augmentation. arXiv:2502.19907, 2025.
3. Li, D., Cheng, Z., Nicholls, G. K., Kong, Q. De-Linearizing Agent Traces: Bayesian Inference of Latent Partial Orders for Efficient Execution. arXiv:2602.02806, 2026.
4. Chen, X., Han, X., Hu, J., Ruiz, F., Liu, L. Order Matters: Probabilistic Modeling of Node Sequence for Graph Generation. ICML 2021, PMLR 139:1630–1639.
5. Goldberg, Y., Nivre, J. Training Deterministic Parsers with Non-Deterministic Oracles. TACL 1:403–414, 2013.
6. Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M., Keutzer, K., Gholami, A. An LLM Compiler for Parallel Function Calling. ICML 2024, PMLR 235:24370–24391.
7. Graves, A., Fernández, S., Gomez, F., Schmidhuber, J. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. ICML 2006, 369–376.
8. Godefroid, P. Partial-Order Methods for the Verification of Concurrent Systems. LNCS 1032, Springer, 1996.
9. Bernstein, A. J. Analysis of Programs for Parallel Processing. IEEE Transactions on Electronic Computers EC-15(5):757–763, 1966.
10. Brightwell, G., Winkler, P. Counting Linear Extensions. Order 8(3):225–242, 1991.
11. Trivedi, H., Khot, T., Hartmann, M., Manku, R., et al. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. ACL 2024, 16022–16076.